

Circles: Final MVP Delivery Report

Final Project

10 May 2026

SUBMITTED TO

Software Development Case Study · CSE-2507M

SUBMITTED BY

Alexandr Tyulkov

Contents

1 Product Summary 1

2 Requirements Delivery 1

3 MVP Hypothesis Outcome 3

4 Development Process 3

5 Theory in Practice 4

6 Architecture Delta 4

1 Product Summary

Circles is a web application for Spotify listening-history analytics. It addresses the lack of a permanent, searchable, always-updating view of a user's listening history and statistics. The primary user is an active Spotify listener who wants private personal music history first, with social and discovery features as secondary value.

2 Requirements Delivery

ID	Requirement	Type	Status	Evidence / note	Sprint
US-01	Connect Spotify account through OAuth and track listening history automatically.	FR	Implemented	Auth, credential refresh, and sync workflows exist. Evidence: apps/backend/src/worker/workflows/sync-history.ts; Assignment 4 dashboard and Hatchet screenshots.	S1-S2
US-02	View top artists, albums, and tracks across selectable ranges.	FR	Implemented	Dashboard, library, stats endpoints, and range tabs are implemented. Evidence: apps/backend/src/api/controllers/me.ts; home-dashboard.png.	S3-S8
US-03	Follow users and view recent listening activity.	FR	Partial	Follow/unfollow and following activity endpoints exist. The demo depends on available test users and seeded social data.	S5
US-04	Request Taste DNA, roast, or scene report.	FR	Partial	All three endpoints and UI cards exist. It depends on provider configuration and cached generation, so it is not core MVP evidence.	S6
US-05	Create playlists, manage tracks, and auto-generate weekly playlist.	FR	Partial	Manual playlist CRUD exists. Weekly generation exists as a worker workflow but is not the main demo path.	S6
US-06	See weekly and all-time global leaderboard.	FR	Implemented	Leaderboard endpoint and UI exist with cached aggregation. Evidence: social-leaderboard.png.	S5
US-07	Upload Spotify history ZIP for historical import.	FR	Implemented	Upload, process, status endpoint, and worker exist. Evidence: spotify-import.png showing 54,963 imported tracks.	S6
US-08	View a public profile with stats and top tracks.	FR	Partial	Public profile route and privacy checks exist. Full top-track profile detail is less complete than the private dashboard.	S5-S8
US-09	See a music match score with another user.	FR	Partial	Backend match endpoint and frontend card exist. Value depends on multiple users with enough shared history.	S5
US-10	Browse a time-machine view for a past date.	FR	Implemented	Time-machine endpoint and authenticated route exist. Evidence: apps/backend/src/api/controllers/me.ts; apps/frontend/src/routes/	S4-S8

ID	Requirement	Type	Status	Evidence / note	Sprint
				<code>_authenticated.time-machine.tsx</code> .	
NFR-P1	95% of API requests within 200 ms under normal load.	NFR	Partial	Read paths are simple and indexed, but no final production load report is attached.	S1/S9
NFR-P2	Spotify sync completes within 30 seconds for one user.	NFR	Partial	Hatchet workflow evidence shows successful sync, but final timed measurement is not attached.	S2/S9
NFR-P3	Leaderboard responds within 500 ms using server cache.	NFR	Implemented	Endpoint uses a 5-minute in-process cache in <code>leaderboard.ts</code> .	S5/S9
NFR-P4	Frontend first contentful paint within 2 seconds.	NFR	Partial	Frontend is deployed, but no final Lighthouse result is attached.	S8/S9
NFR-U1	Usable down to 375 px width.	NFR	Partial	Responsive layouts use Tailwind grids and stacked views, but no final mobile audit is attached.	S8
NFR-U2	WCAG 2.1 AA contrast for controls.	NFR	Partial	UI uses the shared component theme, but no final contrast report is attached.	S8
NFR-U3	New user reaches dashboard within 60 seconds after Spotify login.	NFR	Partial	OAuth and dashboard flow exist. Import size and first sync time can delay full value.	S1/S8
NFR-U4	Errors are human-readable.	NFR	Implemented	Controllers throw clear messages and the import UI shows readable failure text.	S3/S8
NFR-S1	Use HTTPS/TLS 1.2+ for browser, API, and external services.	NFR	Implemented	Frontend and API are deployed at HTTPS URLs in Assignment 4.	S10
NFR-S2	Spotify tokens encrypted at rest and never exposed to the client.	NFR	Partial	Tokens are server-owned and not returned by API responses. Encryption-at-rest proof is not attached.	S1/S9
NFR-S3	Protected API routes validate session and return 401 when unauthenticated.	NFR	Implemented	Authenticated controllers use <code>authMiddleware</code> ; protected frontend routes require session.	S1/S9
NFR-S4	User data inaccessible unless account is public.	NFR	Partial	Public profile controllers reject private profiles. Some social surfaces still need broader privacy tests.	S5/S9
NFR-R1	Maintain at least 99.5% monthly uptime.	NFR	Partial	Deployed app and health checks exist, but no month-long uptime report is attached.	S10
NFR-R2	Token refresh job runs at least every 50 minutes.	NFR	Implemented	Refresh workflow exists and Hatchet evidence shows background jobs running.	S2/S9
NFR-R3	Failed background jobs retry up to 3 times with backoff.	NFR	Implemented	Retry helper and workflow patterns are tested in backend specs.	S2/S6/S9
NFR-R4	Handle Spotify 429 with backoff and retry.	NFR	Partial	Retry support exists. Final Spotify rate-limit simulation is not attached.	S2/S9

ID	Requirement	Type	Status	Evidence / note	Sprint
NFR-SC1	Support at least 10 million scrobbles without query degradation.	NFR	Partial	Schema has user/date-oriented access paths, but no 10 million row benchmark is attached.	S1/S9
NFR-SC2	Workers can scale horizontally without duplicate job execution.	NFR	Implemented	Hatchet owns workflow execution and prevents duplicate job handling.	S2/S9
NFR-SC3	API server is stateless behind a load balancer.	NFR	Implemented	Session and app data live outside API process in shared stores.	S1/S9

Final count: 12 requirements are implemented, 15 are partial, and none are descoped. The delivered product is a working MVP for Spotify login, history sync, dashboard statistics, history browsing, import, leaderboard, and time-machine use. The main gaps are proof gaps for non-functional targets and secondary feature depth for social, generated reports, playlists, public profiles, and matching.

3 MVP Hypothesis Outcome

Question	Answer
State the original hypothesis	If Circles connects to Spotify and syncs recently played tracks every 15 minutes, active Spotify users will return to view their updated listening history because they want a more complete record than Spotify's default history.
What evidence supports it?	The deployed prototype connects to Spotify, syncs data through Hatchet, shows dashboard statistics, and lists chronological history. The Assignment 4 screenshots show dashboard, history, library, import completion, and workflow runs. The Telegram feedback prompt asks testers whether history was useful and whether they would return after automatic updates.
What did not validate?	Repeat retention was not measured over a full week. Feedback was informal and small. It supports the usefulness of the history view, but it does not prove product-market fit.
What changed as a result?	The product focus narrowed to private listening history and sync reliability. Social comparison and generated reports stayed secondary because they do not validate the riskiest assumption as directly as updated history does.

4 Development Process

Item	Evidence
Methodology	Scrum was selected in Assignment 2. In practice it was applied as short weekly planning and review checkpoints, not full team ceremonies.
GitHub	Work is distributed across the course. Examples: 77f5c63 on 2026-03-22 added Assignment 1; 027dd65 on 2026-04-03 added Assignment 2; 0fa3b94 on 2026-04-24 added Assignment 3; f63f33b on 2026-05-10 added Assignment 4; 36415ac and later May commits added deployment and fixes.
Ceremonies / artefacts	Sprint planning happened through weekly task selection. Review happened through assignment submissions and prototype checks. Retrospective was informal because this was a solo project.
What was learned	The sprint cadence helped keep the Spotify sync and history MVP ahead of secondary features. The main process change would be to collect tester feedback earlier, not only after the prototype was already broad.

5 Theory in Practice

The most visible course concept is the Minimum Viable Product from Eric Ries, *The Lean Startup*, in the Build-Measure-Learn discussion. The specific project moment was Assignment 4, where the MVP hypothesis was narrowed to Spotify login, scheduled history sync, storage, and visible listening history.

This helped because the prototype could test one risky assumption instead of trying to prove every planned feature at once. The cost was that some richer features, such as generated reports, playlists, public profiles, and music matching, remained weaker than the core history flow.

6 Architecture Delta

Component / decision	Original design	Final implementation
Deployment	Frontend, API, worker, PostgreSQL, Hatchet, and object storage as separate runtime concerns.	Frontend is deployed on Cloudflare Workers. API is deployed separately on a VPS. Hatchet remains the background workflow runtime.
MVP scope	Full platform with analytics, social, generated reports, playlists, import, profiles, matching, and time-machine.	Core proof is Spotify sync plus history/statistics. Secondary features exist but are not all equally complete.
Generated reports	Part of the advanced feature set.	Implemented as synchronous API calls with weekly cache. It should move further into the worker path if usage grows.
Leaderboard cache	Server-side caching with 5-minute TTL.	Implemented as in-process cache. This works for one API instance but would need shared cache for multiple replicas.